

Лабораторная работа

OpenAI API — POST /v1/responses

Цель работы

Научиться работать с современным Responses API OpenAI и экспериментально исследовать:

- input;
- instructions;
- model;
- max_output_tokens;
- temperature;
- top_p;
- reasoning;
- text.verbosity;
- Structured Output;
- previous_response_id;
- conversation state;
- tools;
- tool_choice;
- parallel_tool_calls;
- built-in tools;
- multimodal input;
- streaming;
- token usage;
- prompt caching;
- background execution.

Главная цель — понять отличие:

Chat Completions:

messages → choices → message

Responses:

input → response → output items

Responses API представляет ответ не просто как текст, а как последовательность типизированных объектов.

1. Endpoint

POST

<https://api.openai.com/v1/responses>

HTTP-заголовки:

Authorization: Bearer \$OPENAI_API_KEY Content-Type: application/json

2. Первый запрос

```
curl https://api.openai.com/v1/responses
-H "Content-Type: application/json"
-H "Authorization: Bearer $OPENAI_API_KEY"
-d '{
  "model": "gpt-5.6",
  "input": "Объясни понятие REST API одним предложением."
}'
```

Минимальная структура:

```
{
  "model": "gpt-5.6",
  "input": "Что такое REST API?"
}
```

В Chat Completions аналог выглядел бы примерно так:

```
{
  "model": "gpt-5.6",
  "messages": [
    {
      "role": "user",
      "content": "Что такое REST API?"
    }
  ]
}
```

Первое принципиальное отличие:

Chat Completions	Responses
messages	input
↓	↓
model	model
↓	↓
choices[]	output[]

↓	↓
message	typed items
↓	↓
content	output_text

3. Исследуем структуру Response

Упрощённый пример:

```
{
  "id": "resp_...",
  "object": "response",
  "created_at": "...",
  "status": "completed",
  "model": "gpt-5.6",
  "output": [
    {
      "type": "message",
      "id": "msg_...",
      "status": "completed",
      "role": "assistant",
      "content": [
        {
          "type": "output_text",
          "text": "...",
          "annotations": []
        }
      ]
    }
  ],
  "usage": {
    "input_tokens": 15,
    "output_tokens": 30,
    "total_tokens": 45
  }
}
```

Основные поля верхнего уровня:

```
id
object
created_at
completed_at
status
error
incomplete_details
```

```
model
output
usage
instructions
metadata
previous_response_id
conversation
reasoning
tools
tool_choice
parallel_tool_calls
temperature
top_p
text
max_output_tokens
service_tier
```

4. id

Пример:

```
"id": "resp_abc123"
```

Уникальный ID Response.

Он создаётся сервером.

Изменить его входным параметром нельзя.

Он используется в том числе для:

```
GET /v1/responses/{response_id}
```

и для продолжения диалога:

```
"previous_response_id": "resp_abc123"
```

5. object

```
"object": "response"
```

Тип объекта.

Для Responses API:

```
object = response
```

В Chat Completions:

```
object = chat.completion
```

6. created_at и completed_at

```
"created_at": 1780000000
```

Unix timestamp создания Response.

При успешно завершённом Response также может присутствовать:

```
"completed_at": 1780000002
```

Изменяются сервером.

7. status

Возможные состояния:

```
completed  
failed  
in_progress  
cancelled  
queued  
incomplete
```

Для обычного синхронного запроса студент чаще всего увидит:

```
"status": "completed"
```

8. input

Это один из важнейших параметров Responses API.

Простейший вариант:

```
{
  "input": "Что такое Kubernetes?"
}
```

В отличие от Chat Completions необязательно создавать массив `messages`.

9. Input как сообщения

Можно использовать более сложную структуру:

```
{
  "input": [
    {
      "role": "user",
      "content": "Что такое Kubernetes?"
    }
  ]
}
```

Можно передать историю:

```
{
  "input": [
    {
      "role": "user",
      "content": "Меня зовут Алексей."
    },
    {
      "role": "assistant",
      "content": "Приятно познакомиться, Алексей."
    },
    {
      "role": "user",
      "content": "Как меня зовут?"
    }
  ]
}
```

10. Эксперимент №1 — input

Выполните:

```
{
  "model": "gpt-5.6",
  "input": "Назови три преимущества микросервисной архитектуры."
}
```

Затем замените `input` :

```
{
  "model": "gpt-5.6",
  "input": "Назови три недостатка микросервисной архитектуры."
}
```

Сравните:

```
output
usage.input_tokens
usage.output_tokens
```

11. instructions

В Responses API инструкции верхнего уровня можно задавать отдельно:

```
{
  "model": "gpt-5.6",
  "instructions": "Ты преподаватель распределённых систем. Отвечай кратко.",
  "input": "Что такое Kafka?"
}
```

`instructions` выполняет функцию system/developer instruction.

Важная особенность:

при использовании `previous_response_id` инструкции предыдущего Response автоматически не переносятся.

То есть:

```
Response 1
instructions = "Отвечай как преподаватель"
```

↓

```
previous_response_id
```

↓

```
Response 2
```

не означает автоматического повторного применения тех же instructions.

Если они нужны — передавайте их снова.

12. Эксперимент №2 — instructions

Запрос А:

```
{
  "model": "gpt-5.6",
  "instructions": "Отвечай как профессор Computer Science.",
  "input": "Что такое TCP?"
}
```

Запрос В:

```
{
  "model": "gpt-5.6",
  "instructions": "Объясняй как десятилетнему ребёнку.",
  "input": "Что такое TCP?"
}
```

Сравните `output`.

Контролируем:

```
instructions
  ↓
style / terminology / complexity
  ↓
output
```

13. model

```
{  
  "model": "gpt-5.6"  
}
```

Определяет используемую модель.

Изменение модели может влиять на:

```
качество  
latency  
стоимость  
reasoning  
доступные tools  
доступные параметры  
context window  
max output
```

Не следует делать вывод, что каждый параметр Responses API поддерживается каждой моделью.

Перед экспериментом необходимо проверить документацию конкретной модели.

14. Эксперимент №3 — model

Выполните один и тот же prompt на двух доступных моделях.

Например:

```
model A  
model B
```

Prompt:

```
Есть сервис заказов, сервис оплаты и сервис склада.  
Предложи механизм обеспечения согласованности данных без распределённой  
транзакции.
```

Зафиксируйте:

```
model  
response time
```

```
output_tokens
reasoning_tokens
качество ответа
```

15. max_output_tokens

Пример:

```
{
  "model": "gpt-5.6",
  "input": "Подробно объясни архитектуру Kubernetes.",
  "max_output_tokens": 100
}
```

Определяет максимальное число генерируемых токенов.

Важно:

лимит включает не только видимый ответ, но и reasoning tokens у reasoning-моделей.

Измените:

```
100
300
1000
```

Сравните:

```
status
incomplete_details
usage.output_tokens
output
```

16. status = incomplete

Если модели не хватило разрешённого бюджета вывода, можно получить:

```
"status": "incomplete"
```

и:

```
"incomplete_details": {  
  "reason": "max_output_tokens"  
}
```

Это принципиально отличается от привычной проверки только текста ответа.

17. temperature

```
{  
  "temperature": 0.2  
}
```

Диапазон:

0 ... 2

Низкое значение:

более стабильный
более сфокусированный
менее разнообразный ответ

Высокое:

больше вариативности
больше случайности

18. Эксперимент №4 — temperature

Используйте prompt:

Придумай название для облачной платформы разработки AI-агентов.

Выполните каждый вариант минимум пять раз.

```
temperature = 0.1
temperature = 0.8
temperature = 1.5
```

Сравните разнообразие результатов.

Важно:

низкая temperature не является гарантией абсолютно идентичного результата.

19. top_p

Пример:

```
{
  "top_p": 0.2
}
```

Это nucleus sampling.

Модель рассматривает наиболее вероятные токены, пока их суммарная вероятность не достигнет `top_p`.

Например:

```
top_p = 0.1
```

означает использование токенов из верхних примерно 10% вероятностной массы.

Не рекомендуется одновременно активно изменять:

```
temperature
и
top_p
```

Лучше экспериментировать с одним из них.

20. reasoning

Для reasoning-моделей:

```
{
  "reasoning": {
    "effort": "low"
  }
}
```

В зависимости от модели могут поддерживаться уровни вроде:

```
none
low
medium
high
xhigh
max
```

Поддержка зависит от конкретной модели.

21. Эксперимент №5 — reasoning.effort

Задача:

В системе есть 5 сервисов.
Каждый имеет availability 99.9%.
Запрос последовательно проходит через все 5 сервисов.
Оцени итоговую availability цепочки.

Выполните:

```
"reasoning": {
  "effort": "low"
}
```

затем:

```
"reasoning": {
  "effort": "medium"
}
```

затем:

```
"reasoning": {  
  "effort": "high"  
}
```

Смотрите:

```
usage.output_tokens_details.reasoning_tokens
```

и сравните:

```
качество  
количество reasoning tokens  
latency  
общий расход tokens
```

22. usage

Пример:

```
{  
  "usage": {  
    "input_tokens": 50,  
    "input_tokens_details": {  
      "cached_tokens": 0,  
      "cache_write_tokens": 0  
    },  
    "output_tokens": 120,  
    "output_tokens_details": {  
      "reasoning_tokens": 60  
    },  
    "total_tokens": 170  
  }  
}
```

Главные показатели:

```
input_tokens  
output_tokens  
total_tokens
```

Для reasoning-моделей отдельно важен:

```
output_tokens_details.reasoning_tokens
```

23. text.verbosity

Responses API позволяет управлять подробностью ответа отдельно от reasoning.

Пример:

```
{
  "model": "gpt-5.6",
  "input": "Объясни паттерн Saga.",
  "text": {
    "verbosity": "low"
  }
}
```

Варианты:

```
low
medium
high
```

24. Эксперимент №6 — verbosity

Выполните:

```
low
medium
high
```

с одним prompt.

Например:

```
Объясни Event-Driven Architecture.
```

Сравните:

длину ответа
output_tokens
число деталей

Очень важное отличие:

reasoning.effort

управляет количеством вычислительного рассуждения,

а

text.verbosity

управляет подробностью видимого ответа.

Это разные параметры.

25. Structured Output

В Responses API настройка формата располагается внутри:

text.format

а не `response_format`, как в Chat Completions.

Пример:

```
{
  "model": "gpt-5.6",
  "input": "Извлеки данные: Иван Петров, 35 лет, Москва.",
  "text": {
    "format": {
      "type": "json_schema",
      "name": "person",
      "strict": true,
      "schema": {
        "type": "object",
        "properties": {
          "name": {
            "type": "string"
          },
          "age": {
```

```
        "type": "integer"
    },
    "city": {
        "type": "string"
    }
},
"required": [
    "name",
    "age",
    "city"
],
"additionalProperties": false
}
}
}
```

Ожидаемый смысл результата:

```
{
  "name": "Иван Петров",
  "age": 35,
  "city": "Москва"
}
```

26. Эксперимент №7 — Structured Output

Возьмите текст:

```
Заказ ORD-1024 создан клиентом Иван Петров.
Стоимость заказа 12500 рублей.
Статус заказа – PAID.
```

Создайте schema:

```
order_id
customer
amount
status
```

Проверьте:

валидность JSON
соответствие типам
наличие обязательных полей

27. output

Это главное архитектурное отличие Responses API.

Chat Completions:

```
choices[]
```

Responses:

```
output[]
```

Но `output` содержит не только сообщения.

В нём могут находиться различные типы элементов:

```
message  
reasoning  
function_call  
web_search_call  
file_search_call  
computer_call  
code_interpreter_call  
MCP-related items  
...
```

Поэтому нельзя писать приложение с предположением:

```
output[0].content[0].text
```

всегда существует.

28. Получение текста

При непосредственном разборе JSON:

```
output[]
  ↓
item.type = message
  ↓
content[]
  ↓
content.type = output_text
  ↓
text
```

В официальных SDK удобнее использовать:

```
response.output_text
```

29. previous_response_id

Одно из важнейших отличий от Chat Completions.

Первый запрос:

```
curl https://api.openai.com/v1/responses
-H "Content-Type: application/json"
-H "Authorization: Bearer $OPENAI_API_KEY"
-d '{
  "model": "gpt-5.6",
  "input": "Меня зовут Алексей, и я изучаю Kubernetes."
}'
```

Получаем:

```
resp_123
```

Второй запрос:

```
{
  "model": "gpt-5.6",
  "previous_response_id": "resp_123",
  "input": "Что я изучаю?"
}
```

Модель может использовать контекст предыдущего Response.

В Chat Completions приложение традиционно заново передавало историю:

```
messages:
  user
  assistant
  user
  assistant
  user
  ...
```

Responses позволяет связать turns через:

```
previous_response_id
```

30. Эксперимент №8 — State

Шаг 1:

```
Меня зовут Андрей.
Я использую PostgreSQL.
Моя система обрабатывает 5000 RPS.
```

Сохраните:

```
response.id
```

Шаг 2:

```
{
  "previous_response_id": "<ID первого response>",
  "input": "Какую базу данных я использую и какая нагрузка?"
}
```

Убедитесь, что модель использовала предыдущий контекст.

31. conversation

Responses API также поддерживает отдельный объект conversation.

```
{
  "conversation": "conv_...",
  "input": "Продолжи обсуждение архитектуры."
}
```

При использовании conversation связанные input/output items автоматически становятся частью conversation.

Важно:

previous_response_id

и

conversation

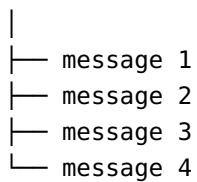
нельзя использовать одновременно.

Это два механизма управления state.

32. State: ключевое сравнение

CHAT COMPLETIONS

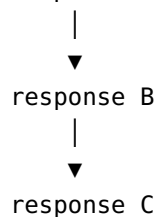
Application



OpenAI API

RESPONSES

response A



или:

```
Conversation
├─ item
├─ item
├─ item
└─ item
```

33. tools

Responses API значительно расширяет модель tools.

Поддерживаются три важные категории:

```
Built-in tools
MCP tools
Custom functions
```

Built-in tools предоставляются OpenAI.

Примеры:

```
web search
file search
code interpreter
computer use
```

Custom function вызывает код вашего приложения.

MCP позволяет модели взаимодействовать с внешними системами через Model Context Protocol.

34. Function Calling

Пример определения собственной функции:

```
{
  "model": "gpt-5.6",
  "input": "Какая погода сейчас в Москве?",
  "tools": [
    {
      "type": "function",
      "name": "get_weather",
```

```

    "description": "Получает текущую погоду для города",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {
          "type": "string"
        }
      },
      "required": [
        "city"
      ],
      "additionalProperties": false
    },
    "strict": true
  }
]
}

```

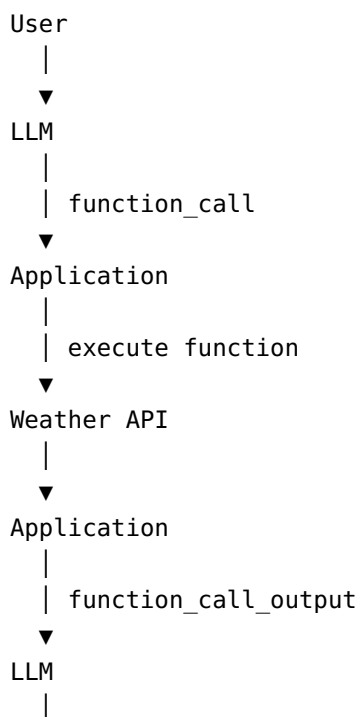
Модель НЕ выполняет вашу функцию.

Она может вернуть вызов:

```
function_call
```

с аргументами.

Архитектура:



▼
Final answer

35. tool_choice

Позволяет контролировать использование tools.

Типичная логика:

```
auto
```

Модель сама решает, нужен ли tool.

Можно ограничивать или принуждать выбор конкретных tools в соответствии с поддерживаемым форматом `tool_choice`.

36. parallel_tool_calls

```
{  
  "parallel_tool_calls": true  
}
```

Позволяет модели создавать несколько tool calls параллельно.

Например:

```
Покажи погоду в Москве, Париже и Берлине.
```

Модель потенциально может создать:

```
get_weather("Москва")  
get_weather("Париж")  
get_weather("Берлин")
```

без последовательного ожидания каждого предыдущего вызова.

37. Эксперимент №9 — Function Calling

Создайте две функции:

```
get_weather(city)
get_exchange_rate(currency)
```

Передайте prompt:

Какая сейчас погода в Москве и курс EUR?

Проверьте:

```
какие tool calls сформированы;
их arguments;
их IDs;
возможность parallel_tool_calls.
```

38. Built-in Web Search

Responses API может использовать OpenAI built-in tools.

Концептуальный запрос:

```
{
  "model": "gpt-5.6",
  "tools": [
    {
      "type": "web_search"
    }
  ],
  "input": "Найди актуальную информацию по заданной теме."
}
```

В `output` может появиться:

```
web_search_call
```

а затем:

```
message
```

То есть:

```
output[0]
```

совсем не обязан быть текстовым message.

39. include

Параметр:

```
"include": [...]
```

позволяет запросить дополнительную информацию.

Например могут запрашиваться:

```
web_search_call.action.sources  
file_search_call.results  
code_interpreter_call.outputs  
message.output_text.logprobs  
reasoning.encrypted_content
```

Это полезно для observability и debugging.

40. Эксперимент №10 — output items

Запустите запрос с tool.

Посмотрите весь:

```
output
```

Для каждого элемента выпишите:

```
type  
id  
status  
content / arguments / result
```

Цель эксперимента:

перестать воспринимать LLM API как:

```
prompt → string
```

Правильная модель:

```
input
  ↓
model
  ↓
sequence of typed events/items
  ↓
application orchestration
```

41. Multimodal Input

Responses API позволяет передавать:

```
text
image
file
```

в рамках общего `input`.

Концептуально:

```
{
  "role": "user",
  "content": [
    {
      "type": "input_text",
      "text": "Что изображено на картинке?"
    },
    {
      "type": "input_image",
      "image_url": "..."
    }
  ]
}
```

То есть модель получает:

TEXT
+
IMAGE

как один input.

42. Эксперимент №11 — Vision

Передайте изображение архитектурной диаграммы.

Prompt:

Проанализируй архитектуру.
Перечисли компоненты и связи между ними.

Затем:

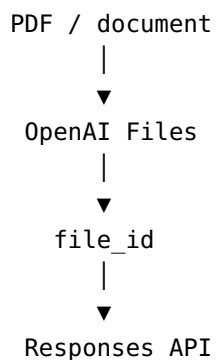
Найди потенциальные Single Points of Failure.

Сравните подход с передачей обычного текста.

43. File Input

Responses API также принимает файлы как input.

Архитектура:



Модель получает документ как часть input.

Практический сценарий:

```
загрузить техническое задание
↓
передать file
↓
попросить извлечь требования
↓
получить structured output
```

44. max_tool_calls

Позволяет ограничить суммарное число вызовов встроенных tools.

Пример:

```
{
  "max_tool_calls": 3
}
```

Это полезный guardrail для агентных сценариев:

```
LLM
↓
tool
↓
LLM
↓
tool
↓
LLM
↓
tool
```

не должен бесконтрольно продолжаться.

45. metadata

Пример:

```
{
  "metadata": {
    "student": "student-42",
    "lab": "responses-api"
  }
}
```

```
}  
}
```

Можно добавить до 16 key/value pairs.

Используется для:

```
tracking  
analytics  
debugging  
filtering  
experiments
```

46. safety_identifier

```
{  
  "safety_identifier": "hashed-user-123"  
}
```

Стабильный идентификатор пользователя приложения.

Не следует передавать PII (Personally Identifiable Information — персональные данные) напрямую.

Вместо email:

```
ivan@example.com
```

используется, например:

```
SHA256(...)
```

47. store

```
{  
  "store": true  
}
```

Определяет возможность хранения Response для последующего retrieval.

По текущей спецификации Responses:

```
store = true
```

используется по умолчанию, если параметр не указан.

Если архитектура требует stateless processing:

```
{  
  "store": false  
}
```

Поведение state/reasoning при этом следует учитывать отдельно.

48. Prompt Caching

Основные параметры:

```
prompt_cache_key  
prompt_cache_options
```

Пример:

```
{  
  "prompt_cache_key": "architecture-course"  
}
```

В usage можно наблюдать:

```
input_tokens_details.cached_tokens  
input_tokens_details.cache_write_tokens
```

Эксперимент имеет смысл проводить на достаточно большом повторяющемся prefix.

49. Эксперимент №12 — Prompt Cache

Создайте большой неизменный instruction/context.

Запрос №1:

```
большой общий context
+
question A
```

Запрос №2:

```
тот же большой context
+
question B
```

Сравните:

```
input_tokens
cached_tokens
cache_write_tokens
latency
```

50. background

Responses может выполняться в background:

```
{
  "background": true
}
```

Это важно для длинных reasoning/agentic задач.

Модель выполнения:

```
POST /responses
|
▼
status = queued/in_progress
|
▼
GET /responses/{id}
|
▼
status = completed
```

51. Streaming

Для streaming:

```
{  
  "stream": true  
}
```

Вместо одного полного JSON клиент получает последовательность событий.

Архитектура:

```
Request  
|  
▼  
response.created  
|  
▼  
output_item.added  
|  
▼  
content_part.added  
|  
▼  
output_text.delta  
|  
▼  
output_text.done  
|  
▼  
response.completed
```

Точные event types зависят от выполняемой операции.

52. Эксперимент №13 — Streaming

Сравните:

```
stream=false
```

и:

```
stream=true
```

Измерьте:

```
Time To First Token/Event  
Time To Complete
```

Пользовательское восприятие latency зачастую определяется именно временем появления первого контента.

53. moderation

Responses API может включать конфигурацию moderation:

```
moderation
```

При её использовании Response может содержать:

```
moderation.input  
moderation.output
```

Это позволяет проверять:

```
вход пользователя  
выход модели
```

54. service_tier

Управляет режимом обработки запроса.

В зависимости от доступности проекта могут существовать варианты:

```
auto  
default  
flex  
priority / fast  
и другие доступные tiers
```

Нельзя рассчитывать, что любой tier доступен любому проекту.

Response показывает фактически применённый:

```
service_tier
```

55. error

Если генерация завершилась ошибкой:

```
"error": {
  "code": "...",
  "message": "...
}
```

Приложение должно анализировать:

```
HTTP status
+
Response error
+
status
```

а не только наличие текста.

56. Главное сравнение API

Chat Completions	Responses
POST /v1/chat/completions	POST /v1/responses
messages	input
system/developer message	instructions или input messages
choices[]	output[]
choices[0].message.content	output items / SDK output_text
max_completion_tokens	max_output_tokens
reasoning_effort	reasoning.effort
verbosity	text.verbosity
response_format	text.format
history through messages	previous_response_id / conversation

Chat Completions	Responses
function tools	function + built-in + MCP tools
primarily message-oriented	item/event-oriented
application controls history	API can maintain state
multimodality via message content	unified multimodal input

57. Самое важное архитектурное отличие

Chat Completions можно мысленно представить:

```

conversation
  ↓
model
  ↓
message

```

Responses:

```

state
  +
input
  +
instructions
  +
tools
  +
reasoning configuration
  |
  ▼
  model
  |
  ▼
typed output items
  |
  ├── reasoning
  ├── tool call
  ├── tool result interaction
  ├── message
  └── annotations

```

Поэтому Responses API значительно ближе к архитектуре AI Agent.

58. Контрольный эксперимент Chat Completions → Responses

Необходимо реализовать одинаковую задачу двумя способами.

Задача:

Ты архитектор информационных систем.

Пользователь:
Мы проектируем интернет-магазин.
Нагрузка – 5000 RPS.
Нужны каталог, заказ, оплата и склад.

Предложи архитектуру системы.
Ответ верни как JSON.

Вариант А

Использовать:

POST /v1/chat/completions

Вариант В

Использовать:

POST /v1/responses

Сравнить:

request JSON
response JSON
структуру system instructions
структуру input
структуру output
Structured Output
token usage
state management
tool integration

59. Итоговая лабораторная задача

Создать мини-агента:

Architecture Assistant

Вход:

описание информационной системы

Агент должен:

1. Проанализировать требования.
2. Определить компоненты.
3. Определить потенциальные риски.
4. При необходимости вызвать tool.
5. Вернуть результат в Structured Output.
6. Сохранить state между двумя turns.

JSON Schema результата:

```
{
  "system_name": "...",
  "components": [
    "...",
  ],
  "risks": [
    "...",
  ],
  "recommendations": [
    "...",
  ]
}
```

Второй turn:

Теперь увеличь предполагаемую нагрузку в 10 раз.
Что необходимо изменить?

Для второго запроса обязательно использовать:

previous_response_id

60. Что студент должен показать преподавателю

Не просто работающий ответ модели.

Необходимо продемонстрировать влияние:

```
input
instructions
model
max_output_tokens
temperature или top_p
reasoning.effort
text.verbosity
text.format
previous_response_id
tools
tool_choice
parallel_tool_calls
stream
```

И объяснить изменение:

```
output
status
usage
reasoning_tokens
tool calls
structured output
state
```

61. Итоговая таблица «что меняем → что наблюдаем»

INPUT PARAMETER	OBSERVE
input	
→ output	
instructions	
→ content/style	

model
→ model/output/usage/latency

max_output_tokens
→ output_tokens/status/incomplete_details

temperature
→ variability

top_p
→ variability

reasoning.effort
→ reasoning_tokens/quality/latency

text.verbosity
→ visible answer length/output_tokens

text.format
→ output structure

previous_response_id
→ conversation context

conversation
→ persistent state

tools
→ output tool items

tool_choice
→ tool selection

parallel_tool_calls
→ parallel calls

max_tool_calls
→ number of tool executions

include
→ additional response information

prompt_cache_key
→ caching behavior

store
→ retrieval/state behavior

stream
→ stream events/TTFT

```
background
  → queued/in_progress/completed

metadata
  → metadata

safety_identifier
  → request attribution

service_tier
  → processing tier
```

62. Главное, что необходимо запомнить

После Chat Completions разработчик привык к модели:

```
messages → model → assistant message
```

После Responses API нужно перейти к модели:

```
INPUT
+
STATE
+
INSTRUCTIONS
+
TOOLS
+
REASONING
  |
  ▼
  LLM
  |
  ▼
OUTPUT ITEMS
```

Это и есть ключевой переход от простого использования LLM как чат-бота к использованию LLM как компонента AI-системы и AI-агента.